Technical Learnings

TECHNICAL KNOWLEDGE BASE

MariaDB

Architecture · Administration · High Availability · Security · Performance

Enterprise Reference & Operations Guide

DOCUMENT CONTROL

Attribute	Detail
Document title	MariaDB — Complete Technical Knowledge Base
Document ID	SDC-KB-MARIADB-001
Version	1.0
Author	Praveen Madupu — Senior SQL Server & Cloud DBA
Publisher	Database Admin
Technology coverage	MariaDB Community Server 10.6 → 12.3 LTS (rolling: 12.x / 13.0)
Latest LTS at publication	MariaDB 12.3 LTS (GA 12.3.2, May 2026)
Audience	DBAs, Platform Engineers, SREs, Cloud Migration Engineers
Classification	Technical — Public / Educational
Prepared as	Senior Technical Architect & Documentation deliverable

Table of Contents

1. Overview 5
1.1 Why MariaDB was forked 5
1.2 MariaDB vs. MySQL — key differences 5
1.3 Editions & release model 6
1.4 Ecosystem at a glance 6
2. Architecture 7
2.1 Layered server architecture 7
2.2 Layer responsibilities 7
2.3 InnoDB internals (default engine) 7
2.4 Threads vs. processes 8
3. Components 9
3.1 Core binaries & tools 9
3.2 Storage engines 9
3.3 Plugins 9
4. Installation 10
4.1 Ubuntu / Debian (APT) 10
4.2 RHEL / Rocky / AlmaLinux (DNF) 10
4.3 Docker 10
4.4 Post-install hardening & verification 11
5. Configuration 12
5.1 Configuration file load order 12
5.2 Example server configuration 12
5.3 Key server variables 13
6. Prerequisites 14
6.1 Hardware sizing guidance 14
6.2 Operating system tuning 14
6.3 Network ports 14
7. Security Best Practices 15
7.1 Authentication & accounts 15
7.2 Encryption in transit (TLS) 15
7.3 Encryption at rest 16
7.4 Auditing 16
7.5 Compliance control mapping 16

8. Performance Tuning ... 17
8.1 Memory & InnoDB ... 17
8.2 Query optimization workflow ... 17
8.3 Key status counters ... 17
9. Monitoring ... 19
9.1 Key metrics to track ... 19
9.2 Native monitoring commands ... 19
10. Logging ... 20
10.1 Log types ... 20
10.2 Binary log formats ... 20
10.3 Configuring & rotating logs ... 20
11. Backup & Recovery ... 22
11.1 Backup methods compared ... 22
11.2 Physical backup with mariabackup ... 22
11.3 Logical backup & restore ... 23
11.4 Point-in-time recovery (PITR) ... 23
12. Disaster Recovery ... 24
12.1 DR strategies by RPO/RTO ... 24
12.2 Reference DR topology ... 24
12.3 Failover runbook (outline) ... 24
13. High Availability ... 25
13.1 HA options compared ... 25
13.2 Asynchronous replication setup ... 25
13.3 Galera Cluster + MaxScale ... 25
13.4 Quorum & split-brain ... 26
14. Troubleshooting ... 27
14.1 Symptom → diagnosis → resolution ... 27
15. Common Errors ... 28
16. Frequently Asked Questions ... 29
17. Interview Questions ... 30
17.1 Fundamentals & architecture ... 30
17.2 Replication & HA ... 30
17.3 Performance & operations ... 30
17.4 Security & backup ... 31
18. Real-world Use Cases ... 32

19. Best Practices 33
20. References 34

1. Overview

MariaDB is an open-source, relational database management system (RDBMS) created in 2009 by **Michael "Monty" Widenius** — the original author of MySQL — as a community-driven, backward-compatible fork of MySQL following Oracle's acquisition of Sun Microsystems. It is named after Monty's daughter Maria (MySQL was named after his daughter My). The project is stewarded by the non-profit **MariaDB Foundation** (governing the open-source server) and developed commercially by **MariaDB plc** (Enterprise offerings).

MariaDB is engineered as a **drop-in replacement for MySQL** for most workloads: it shares the wire protocol, SQL dialect, file formats, client APIs, and connector interfaces, while extending them with additional storage engines, optimizer enhancements, security features, and — in recent releases — native JSON and vector search capabilities. It powers large-scale platforms including Wikipedia, Google's internal systems, and the default database in most major Linux distributions.

1.1 Why MariaDB was forked

- **Open-governance guarantee** — assurance the server remains free and open source (GPLv2) regardless of any single vendor.
- **Feature velocity** — faster adoption of community contributions, new storage engines, and optimizer work.
- **Compatibility** — retain MySQL compatibility so existing applications migrate with minimal or no change.

1.2 MariaDB vs. MySQL — key differences

Dimension	MariaDB	MySQL (Oracle)
License (server)	GPLv2 (fully open)	GPLv2 + proprietary Enterprise
Governance	MariaDB Foundation + plc	Oracle Corporation
Default engine	InnoDB (Aria for system tables)	InnoDB
Extra storage engines	Aria, ColumnStore, Spider, MyRocks, S3, CONNECT	Limited
Clustering	Galera (sync, built-in)	Group Replication / InnoDB Cluster
Threading	Thread-per-connection + thread pool	Thread-per-connection (pool in Enterprise)
JSON	JSON functions + native since 11.x	Native JSON type
Vector search	Native VECTOR type & HNSW (11.8+)	Added in later MySQL 9.x
Connectors license	LGPL (commercial-friendly)	GPL

ℹ **NOTE — Compatibility caveat**
MariaDB and MySQL have diverged since MySQL 5.7. For MySQL 8.0+ features (e.g., certain data-dictionary internals, some JSON syntax, CTE/window edge cases) validate application compatibility before treating MariaDB as a literal drop-in. For 5.5–5.7-era workloads, migration is typically seamless.

1.3 Editions & release model

Community Server (free, GPLv2) is the focus of this document. **MariaDB Enterprise Server** adds hardened builds, extended support, backup tooling, and enterprise features. Releases follow two tracks:

- **LTS (Long-Term Support)** — yearly, maintained 3 years (binaries) + 2 years critical source fixes. Recommended for production.
- **Rolling (GA) releases** — quarterly, cutting-edge features, supported until the next release. Since MariaDB 12.x, the `.3` release of each major series becomes the LTS.

Series	Type	Notable feature	Status (mid-2026)
10.6	LTS	Atomic DDL	EOL ~Jul 2026
10.11	LTS	Optimizer & replication	Supported
11.4	LTS	InnoDB & privilege changes	Supported
11.8	LTS	Native VECTOR type + HNSW	Supported (to 2028)
12.3	LTS	snapshot isolation ON, GIS/JSON	Latest LTS (to Jun 2029)
13.0	Rolling	Procedural SQL, observability	Preview/rolling

1.4 Ecosystem at a glance

◆ DIAGRAM — MariaDB ecosystem & where it fits

```
flowchart TB
  subgraph Clients["Applications & Clients"]
    A1[Web / App Servers]
    A2[BI & Analytics]
    A3[CLI & Connectors\n(C, Java, Python, Node.js, ODBC)]
  end
  subgraph Access["Access / Routing Layer"]
    MX[MaxScale\nProxy · Router · Failover]
  end
  subgraph Data["MariaDB Data Tier"]
    P[(Primary\nmariadbd)]
    R1[(Replica)]
    G[(Galera Cluster\nsync multi-master)]
  end
  subgraph Ops["Operations"]
    BK[mariabackup / mariadb-dump]
    MON[Monitoring\nPMM · Prometheus · Grafana]
  end
  A1 --> MX
  A2 --> MX
  A3 --> MX
  MX --> P
  P --> R1
  MX --> G
  P -.-> BK
  P -.-> MON
```

2. Architecture

MariaDB uses a **multi-layered, pluggable storage-engine architecture**. The server process (`mariadbd`) is multi-threaded (one thread per connection, optionally pooled) rather than multi-process. Client requests flow down through a connection layer, a SQL processing layer (parser, optimizer, executor), and a storage-engine abstraction API to the chosen engine, which manages on-disk and in-memory structures.

2.1 Layered server architecture

◆ DIAGRAM — MariaDB layered server architecture

```
flowchart TB
  C[Clients / Connectors] -->|MySQL wire protocol :3306| CL
  subgraph mariadbd["mariadbd server process"]
    CL[Connection & Thread Layer\nauth · thread pool · session]
    SQL[SQL Layer\nParser -> Optimizer -> Executor\nquery cache · privileges]
    API{{Storage Engine API\n(handler interface)}}
    CL --> SQL --> API
    subgraph ENG["Pluggable Storage Engines"]
      IN[InnoDB\nACID · MVCC]
      AR[Aria\nsystem/temp]
      MR[MyRocks / ColumnStore\nSpider / CONNECT]
    end
    API --> IN
    API --> AR
    API --> MR
  end
  IN --> FS[(Data files · Redo/Undo · Binlog)]
  AR --> FS
  MR --> FS
```

2.2 Layer responsibilities

Layer	Responsibility	Key elements
Connection / thread	Accept connections, authenticate, manage sessions	Thread pool, auth plugins, SSL handshake
SQL layer	Parse, optimize, execute SQL; enforce security	Parser, cost-based optimizer, executor, privileges
Storage-engine API	Abstract table operations from engines	handler / handlerton interface
Storage engines	Store & retrieve rows, indexes, transactions	InnoDB, Aria, MyRocks, ColumnStore, Spider
Physical storage	Persist data & logs to disk	Data files, redo/undo, binlog, temp

2.3 InnoDB internals (default engine)

InnoDB is the default transactional engine, providing **ACID** compliance, **MVCC** (multi-version concurrency control), row-level locking, foreign keys, and crash recovery. Its memory and disk structures are the primary tuning surface.

◆ DIAGRAM — InnoDB memory & disk structures

```
flowchart LR
  subgraph MEM["In-Memory"]
```

```
    BP[Buffer Pool\ndata + index pages]
    CB[Change Buffer]
    LB[Log Buffer]
    AH[Adaptive Hash Index]
  end
  subgraph DISK["On-Disk"]
    TS[(Tablespaces\n.ibd files)]
    RL[(Redo Log\nib_logfile)]
    UL[(Undo Logs)]
    SYS[(System Tablespace)]
  end
  BP <--> TS
  LB --> RL
  BP --> UL
  CB --> TS
  AH -.speeds.-> BP
```

Structure	Location	Purpose
Buffer pool	RAM	Caches data & index pages; the single most important tuning knob
Redo log	Disk	Write-ahead log for crash recovery (durability)
Undo log	Disk	Old row versions for MVCC & rollback
Change buffer	RAM	Buffers secondary-index changes to non-resident pages
Doublewrite buffer	Disk	Guards against torn/partial page writes
Log buffer	RAM	Buffers redo entries before flush to redo log

2.4 Threads vs. processes

> ℹ **NOTE — Design choice**
> MariaDB (like MySQL) uses **threads, not processes**, for connections. This lowers context-switch cost and memory overhead versus process-per-connection systems (e.g., PostgreSQL). Under very high connection counts, enable the `thread_pool` to bound active threads and avoid thread thrashing.

3. Components

The MariaDB platform is a set of cooperating binaries, engines, plugins, and companion tools. Note that legacy mysql-prefixed binaries were renamed to mariadb-* (symlinks preserved for compatibility).

3.1 Core binaries & tools

Component	Binary	Role
Server daemon	`mariadbd`	Core database engine (was mysqld)
Interactive client	`mariadb`	CLI SQL client (was mysql)
Logical backup	`mariadb-dump`	SQL/logical dump (was mysqldump)
Physical backup	`mariabackup`	Hot physical backup for InnoDB/Aria
Admin utility	`mariadb-admin`	Status, shutdown, flush, ping (was mysqladmin)
Upgrade tool	`mariadb-upgrade`	Upgrades system tables after version change
Import/check	`mariadb-import / mariadb-check`	Bulk load; check/repair/optimize tables
Binlog utility	`mariadb-binlog`	Read/replay binary logs for PITR
Secure setup	`mariadb-secure-installation`	Post-install hardening script
Proxy / router	`MaxScale`	Query routing, load balancing, auto-failover
Cluster library	`galera (wsrep)`	Synchronous multi-master replication provider

3.2 Storage engines

Engine	Type	Best for
InnoDB	Transactional (ACID, MVCC)	General-purpose OLTP (default)
Aria	Crash-safe non-transactional	Internal/system & temp tables
MyISAM	Legacy non-transactional	Read-heavy legacy (avoid for new work)
ColumnStore	Columnar, distributed	Analytics / OLAP at scale
MyRocks	LSM-tree (RocksDB)	Write-heavy, space-efficient workloads
Spider	Sharding / federation	Horizontal partitioning across nodes
CONNECT	External data	Query CSV, XML, ODBC sources as tables
MEMORY	In-RAM	Volatile lookups / temp caches

3.3 Plugins

- **Authentication:** `unix_socket`, `ed25519`, `mysql_native_password`, `gssapi`, PAM.
- **Audit:** `server_audit` — records connections, queries, and admin actions for compliance.
- **Encryption:** key-management plugins (`file_key_management`, KMS/AWS) for data-at-rest encryption.

4. Installation

MariaDB installs from official OS repositories, the vendor repository (recommended for latest LTS), binary tarballs, Docker, or Windows MSI. Prefer the official MariaDB repository to control the exact version.

◆ DIAGRAM — Installation method decision flow

```
flowchart TD
  Start([Need MariaDB]) --> Q1{Environment?}
  Q1 -->|Linux server, prod| Q2{Need latest LTS?}
  Q2 -->|Yes| REPO[Official MariaDB repo\napt / dnf]
  Q2 -->|Distro default OK| OS[OS package manager]
  Q1 -->|Container / CI| DOCKER[Docker image]
  Q1 -->|Windows| MSI[MSI installer]
  Q1 -->|Custom / air-gapped| TAR[Binary tarball]
  REPO --> Secure[mariadb-secure-installation]
  OS --> Secure
  DOCKER --> Verify([Verify & connect])
  MSI --> Verify
  TAR --> Secure --> Verify
```

4.1 Ubuntu / Debian (APT)

BASH

```
# Add the official MariaDB repository (pin to an LTS, e.g. 12.3)
curl -LsS https://r.mariadb.com/downloads/mariadb_repo_setup | \
  sudo bash -s -- --mariadb-server-version="mariadb-12.3"

sudo apt-get update
sudo apt-get install -y mariadb-server mariadb-client mariadb-backup

sudo systemctl enable --now mariadb
sudo systemctl status mariadb
```

4.2 RHEL / Rocky / AlmaLinux (DNF)

BASH

```
# Configure the vendor repo and install
curl -LsS https://r.mariadb.com/downloads/mariadb_repo_setup | \
  sudo bash -s -- --mariadb-server-version="mariadb-12.3"

sudo dnf install -y MariaDB-server MariaDB-client MariaDB-backup
sudo systemctl enable --now mariadb
```

4.3 Docker

BASH

```
docker run -d --name mariadb \
  -e MARIADB_ROOT_PASSWORD='StrongP@ss!' \
  -e MARIADB_DATABASE=appdb \
  -e MARIADB_USER=appuser -e MARIADB_PASSWORD='AppP@ss!' \
  -p 3306:3306 \
  -v mariadb_data:/var/lib/mysql \
  mariadb:12.3
```

```
docker exec -it mariadb mariadb -uroot -p
```

4.4 Post-install hardening & verification

BASH

```
# Interactive hardening: set root auth, remove anon users,
# disallow remote root, drop the test database, reload privileges.
sudo mariadb-secure-installation

# Verify
mariadb --version
sudo mariadb -e "SELECT VERSION(); SHOW ENGINES\G"
```

✓ **BEST PRACTICE — Always run secure-installation**

On production hosts, run mariadb-secure-installation immediately after install. It removes anonymous accounts and the test database, and blocks remote root login — the three most common causes of easy compromise.

5. Configuration

MariaDB reads configuration from `my.cnf` / `my.ini` files and included directories. On Linux the common locations and load order are read top-to-bottom, with later files overriding earlier ones.

5.1 Configuration file load order

Order	Path	Scope
1	`/etc/mysql/my.cnf`	Global (Debian/Ubuntu)
2	`/etc/my.cnf`	Global (RHEL family)
3	`/etc/my.cnf.d/*.cnf`	Drop-in fragments
4	`/etc/mysql/mariadb.conf.d/*.cnf`	Distro drop-ins
5	`~/.my.cnf`	Per-user (client) overrides

5.2 Example server configuration

INI

```
# /etc/my.cnf.d/60-server-tuning.cnf
[mariadbd]
# --- Core ---
bind_address              = 0.0.0.0
max_connections           = 500
thread_handling           = pool-of-threads   # enable thread pool

# --- InnoDB (primary tuning surface) ---
innodb_buffer_pool_size   = 12G               # ~70% of RAM on a dedicated host
innodb_buffer_pool_instances = 8
innodb_log_file_size      = 2G
innodb_flush_log_at_trx_commit = 1            # 1 = full ACID durability
innodb_flush_method       = O_DIRECT
innodb_file_per_table     = ON

# --- Logging ---
slow_query_log            = ON
slow_query_log_file       = /var/log/mariadb/slow.log
long_query_time           = 1
log_error                 = /var/log/mariadb/error.log

# --- Binary log (replication + PITR) ---
log_bin                   = /var/lib/mysql/binlog
binlog_format             = ROW
server_id                 = 1
expire_logs_days          = 7

[client]
default-character-set     = utf8mb4
```

5.3 Key server variables

Variable	Typical value	Effect
`innodb_buffer_pool_size`	60–80% RAM	Cache size; biggest performance lever
`innodb_log_file_size`	512M–4G	Redo capacity; larger = fewer checkpoints
`innodb_flush_log_at_trx_commit`	1 / 2 / 0	Durability vs. throughput trade-off
`max_connections`	150–1000	Concurrent client cap
`tmp_table_size / max_heap_table_size`	64M+	In-memory temp table size before disk spill
`thread_handling`	pool-of-threads	Bounds active threads at high concurrency
`innodb_io_capacity`	200–2000	Background I/O rate (match storage IOPS)

⚠ **WARNING — Dynamic vs. static**

Many variables are dynamic (`SET GLOBAL innodb_io_capacity=1000;`) and take effect immediately, but reset on restart. Persist changes in a config file, and note that `innodb_buffer_pool_size` is resizable online but `innodb_log_file_size` changes require care. Use `SET PERSIST` where supported to write to a persisted config.

6. Prerequisites

Before deploying MariaDB in production, provision hardware, tune the OS, and open the required network ports. The guidance below suits a dedicated database host.

6.1 Hardware sizing guidance

Tier	vCPU	RAM	Storage	Notes
Dev / test	2–4	8–16 GB	SSD 50 GB	Non-critical, single node
Small OLTP	4–8	32–64 GB	NVMe 200 GB+	Buffer pool ~70% RAM
Production OLTP	8–16	64–256 GB	NVMe RAID10	Replica(s) + backups
Analytics (ColumnStore)	16+	128 GB+	Large NVMe/S3	Columnar, scan-heavy

6.2 Operating system tuning

Setting	Recommended	Why
Filesystem	XFS or ext4	Stable, good for DB I/O patterns
`vm.swappiness`	1–10	Avoid swapping the buffer pool
Transparent Huge Pages	disabled	THP causes latency spikes for InnoDB
I/O scheduler	none/mq-deadline (NVMe)	Lower latency on fast storage
open files (nofile)	≥ 65535	Avoid 'too many open files' errors
NUMA	interleave / pin	Consistent memory access for buffer pool

BASH

```
# Raise file-descriptor limit for the mariadb service
sudo mkdir -p /etc/systemd/system/mariadb.service.d
cat <<'EOF' | sudo tee /etc/systemd/system/mariadb.service.d/limits.conf
[Service]
LimitNOFILE=100000
EOF
# Disable Transparent Huge Pages at runtime
echo never | sudo tee /sys/kernel/mm/transparent_hugepage/enabled

sudo sysctl -w vm.swappiness=1
sudo systemctl daemon-reload && sudo systemctl restart mariadb
```

6.3 Network ports

Port	Protocol	Used by
3306	TCP	Client / SQL connections
4567	TCP/UDP	Galera replication traffic
4568	TCP	Galera Incremental State Transfer (IST)
4444	TCP	Galera State Snapshot Transfer (SST)
6603 / 8989	TCP	MaxScale admin / GUI (configurable)

7. Security Best Practices

Database security is layered: harden the host and network, enforce strong authentication and least-privilege access, encrypt data in transit and at rest, and audit activity. The model below maps directly to SOX and PCI-DSS control expectations.

◆ DIAGRAM — Defense-in-depth for MariaDB

```
flowchart TB
  subgraph L1[Network Layer]
    FW[Firewall / Security Group\nrestrict 3306 to app subnets]
    TLS[TLS in transit]
  end
  subgraph L2[Authentication]
    AUTH[Strong auth plugins\ned25519 / unix_socket]
    PWD[Password validation]
  end
  subgraph L3[Authorization]
    RBAC[Roles + least privilege]
    SCHEMA[Object-level GRANTs]
  end
  subgraph L4[Data Protection]
    ENC[Encryption at rest\nkeyring plugin]
    MASK[Masking / limited exposure]
  end
  subgraph L5[Audit & Monitoring]
    AUD[server_audit plugin]
    LOG[Centralized logs / SIEM]
  end
  FW --> AUTH --> RBAC --> ENC --> AUD
```

7.1 Authentication & accounts

- Prefer `ed25519` or `unix_socket` over legacy password hashes; avoid `mysql_native_password` where possible.
- Enforce complexity with the password-validation plugin; set `password_reuse_check` and expiry policies.
- Remove anonymous accounts and the test database; never allow remote root login.

SQL

```
-- Create a least-privilege application user with ed25519
CREATE USER 'appuser'@'10.0.%'
  IDENTIFIED VIA ed25519 USING PASSWORD('S3cure!Pass');

-- Role-based access (RBAC)
CREATE ROLE app_rw;
GRANT SELECT, INSERT, UPDATE, DELETE ON appdb.* TO app_rw;
GRANT app_rw TO 'appuser'@'10.0.%';
SET DEFAULT ROLE app_rw FOR 'appuser'@'10.0.%';
FLUSH PRIVILEGES;
```

7.2 Encryption in transit (TLS)

INI

```
# Server config
[mariadbd]
ssl_cert = /etc/mysql/certs/server-cert.pem
```

```
ssl_key  = /etc/mysql/certs/server-key.pem
ssl_ca   = /etc/mysql/certs/ca-cert.pem
require_secure_transport = ON     # reject non-TLS connections
```

SQL

```
-- Force TLS for a specific user
ALTER USER 'appuser'@'10.0.%' REQUIRE SSL;
-- Verify from client
SHOW STATUS LIKE 'Ssl_cipher';
```

7.3 Encryption at rest

INI

```
[mariadbd]
plugin_load_add          = file_key_management
file_key_management_filename = /etc/mysql/keys/keyfile.enc
file_key_management_encryption_algorithm = AES_CTR
innodb_encrypt_tables    = ON
innodb_encrypt_log       = ON
innodb_encryption_threads = 4
```

7.4 Auditing

SQL

```
INSTALL SONAME 'server_audit';
SET GLOBAL server_audit_logging = ON;
SET GLOBAL server_audit_events  = 'CONNECT,QUERY,TABLE';
SET GLOBAL server_audit_file_path = '/var/log/mariadb/audit.log';
```

7.5 Compliance control mapping

Control area	SOX	PCI-DSS	MariaDB mechanism
Access control	✓	Req 7/8	Roles, least-privilege GRANTs, auth plugins
Encryption in transit	✓	Req 4	TLS / require_secure_transport
Encryption at rest	✓	Req 3	file_key_management / KMS
Audit trail	✓	Req 10	server_audit plugin + SIEM
Change tracking	✓	Req 10	Binary log + audit log retention

CRITICAL — Never store secrets in plaintext configs

Keep TLS keys and encryption key files outside world-readable paths (chmod 600, owned by the mysql user). For regulated environments, integrate an external KMS/HSM rather than a local key file.

8. Performance Tuning

Performance work proceeds top-down: right-size memory, then optimize queries and indexing, then tune I/O and concurrency. Measure before and after every change using the slow query log and status counters.

8.1 Memory & InnoDB

Lever	Guidance
`innodb_buffer_pool_size`	Set to ~60–80% of RAM on a dedicated host; target a high buffer-pool hit ratio (>99%).
`innodb_log_file_size`	Size so redo covers ~1 hour of writes; reduces checkpoint churn.
`innodb_flush_log_at_trx_commit`	1 for full durability; 2 for more throughput with slight risk on OS crash.
`innodb_io_capacity(_max)`	Match to underlying storage IOPS so background flushing keeps up.
Redo & doublewrite	On NVMe, keep doublewrite ON unless the filesystem guarantees atomic writes.

8.2 Query optimization workflow

◆ DIAGRAM — Query tuning loop

```
flowchart LR
  A[Capture slow queries\nslow_query_log] --> B[EXPLAIN / ANALYZE]
  B --> C{Full scan or\nbad join order?}
  C -->|Yes| D[Add / fix indexes\nrewrite query]
  C -->|No| E[Check locks / temp tables]
  D --> F[Re-measure]
  E --> F
  F -->|Improved| G[Deploy]
  F -->|No| B
```

SQL

```
-- Find the plan and real execution profile
EXPLAIN SELECT * FROM orders WHERE customer_id = 42 AND status = 'OPEN';
ANALYZE FORMAT=JSON
  SELECT * FROM orders WHERE customer_id = 42 AND status = 'OPEN';

-- Composite index to serve the predicate
CREATE INDEX ix_orders_cust_status ON orders (customer_id, status);

-- Live engine diagnostics
SHOW ENGINE INNODB STATUS\G
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
```

8.3 Key status counters

Counter	Watch for	Meaning
`Innodb_buffer_pool_reads`	Rising fast	Pages read from disk — buffer pool too small
`Threads_running`	High & sustained	Concurrency pressure / contention
`Created_tmp_disk_tables`	High ratio	Temp tables spilling to disk — tune tmp sizes / rewrite
`Handler_read_rnd_next`	Very high	Table scans — missing indexes

Counter	Watch for	Meaning
`Aborted_connects`	Rising	Auth/network problems

✓ **BEST PRACTICE — Change one thing at a time**

Tune iteratively and measure with a representative workload. Bulk-changing many variables at once makes regressions impossible to attribute.

9. Monitoring

Effective monitoring combines built-in instrumentation (status variables, performance_schema, information_schema) with an external metrics stack for dashboards, alerting, and historical trend analysis.

◆ DIAGRAM — Monitoring & alerting stack

```
flowchart LR
  DB[(MariaDB nodes)] --> EXP[mysqld_exporter]
  EXP --> PROM[Prometheus\nscrape + store]
  PROM --> GRAF[Grafana\ndashboards]
  PROM --> ALERT[Alertmanager\nthresholds]
  DB -. native .-> PS[performance_schema\ninformation_schema]
  ALERT --> ONCALL[On-call / Slack / PagerDuty]
  GRAF --> TEAM[DBA / SRE]
```

9.1 Key metrics to track

Category	Metric	Alert guidance
Availability	Uptime, can-connect	Page on connection failure
Throughput	Questions/s, QPS, TPS	Trend vs. baseline
Buffer pool	Hit ratio	Warn < 99%
Connections	Threads_connected vs. max_connections	Warn > 80% of max
Replication	Seconds_Behind_Master	Warn > 30s, page > 300s
Locks	Lock waits, deadlocks	Investigate spikes
Disk	Free space on data volume	Page < 15% free

9.2 Native monitoring commands

SQL

```
-- Global counters & live sessions
SHOW GLOBAL STATUS;
SHOW GLOBAL VARIABLES LIKE 'max_connections';
SHOW PROCESSLIST;                  -- or: SHOW FULL PROCESSLIST;
SELECT * FROM information_schema.PROCESSLIST WHERE COMMAND <> 'Sleep';

-- Replication health
SHOW REPLICA STATUS\G            -- (SHOW SLAVE STATUS on older versions)

-- Performance schema: top statements by latency
SELECT DIGEST_TEXT, COUNT_STAR, ROUND(SUM_TIMER_WAIT/1e12,2) AS total_s
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC LIMIT 10;
```

ℹ NOTE — Tooling options

Common stacks include Percona Monitoring & Management (PMM), Prometheus + mysqld_exporter + Grafana, and enterprise suites (Zabbix, Nagios, Datadog). PMM is the fastest path to rich MariaDB/MySQL dashboards.

10. Logging

MariaDB emits several log streams, each serving a distinct operational purpose. Configure paths, verbosity, and rotation deliberately — logs are essential for troubleshooting, PITR, and audit.

10.1 Log types

Log	Purpose	Key variable
Error log	Startup, shutdown, crashes, warnings	`log_error`
General query log	Every statement (debug only — noisy)	`general_log`
Slow query log	Queries over a latency threshold	`slow_query_log`
Binary log	All data changes; replication + PITR	`log_bin`
Relay log	Replicated events on a replica	`relay_log`
Audit log	Security/compliance event trail	`server_audit_file_path`

10.2 Binary log formats

Format	What it logs	Trade-off
ROW	Actual changed rows	Safe & deterministic (recommended)
STATEMENT	The SQL statement	Compact but unsafe for non-deterministic SQL
MIXED	Statement, falling back to row	Balance of size and safety

10.3 Configuring & rotating logs

INI

```
[mariadbd]
log_error        = /var/log/mariadb/error.log
slow_query_log   = ON
slow_query_log_file = /var/log/mariadb/slow.log
long_query_time  = 1
log_slow_verbosity = query_plan,explain
log_bin          = /var/lib/mysql/binlog
binlog_format    = ROW
expire_logs_days = 7               # auto-purge old binlogs
```

BASH

```
# /etc/logrotate.d/mariadb  (error & slow logs)
/var/log/mariadb/*.log {
    weekly
    rotate 8
    compress
    missingok
    notifempty
    sharedscripts
    postrotate
      /usr/bin/mariadb-admin flush-logs 2>/dev/null || true
    endscript
```

```
}
```

> ⚠ **WARNING — Never rotate binlogs with logrotate**
> Manage binary logs with `expire_logs_days` / `binlog_expire_logs_seconds` or `PURGE BINARY LOGS` — never delete them with logrotate or `rm`, which can break replicas and destroy point-in-time recovery capability.

11. Backup & Recovery

A resilient backup strategy layers physical backups (fast, whole-instance restore), logical backups (portable, granular), and binary logs (point-in-time recovery). Combine full + incremental physical backups with continuous binlog archiving.

◆ DIAGRAM — Layered backup & recovery strategy

```
flowchart TB
  subgraph Sources
    DB[(MariaDB Primary)]
  end
  DB --> FULL[Weekly FULL\nmariabackup]
  DB --> INCR[Daily INCREMENTAL\nmariabackup --incremental]
  DB --> BINLOG[Continuous binlog\narchive]
  DB --> LOGICAL[Periodic logical\nmariadb-dump / mydumper]
  FULL --> STORE[(Offsite / Object storage\n3-2-1 rule)]
  INCR --> STORE
  BINLOG --> STORE
  LOGICAL --> STORE
  STORE --> RESTORE[[Restore + replay binlog\n= Point-in-Time Recovery]]
```

11.1 Backup methods compared

Method	Type	Pros	Cons
`mariabackup`	Physical (hot)	Fast full/incremental, low overhead	Whole-instance granularity
`mariadb-dump`	Logical	Portable, per-object, human-readable	Slow to restore large DBs
mydumper/myloader	Logical (parallel)	Fast parallel logical dump/restore	Extra tool to manage
Snapshot (LVM/EBS)	Storage-level	Near-instant, whole-volume	Needs FLUSH/consistency care

11.2 Physical backup with mariabackup

BASH

```
# FULL backup
mariabackup --backup \
  --target-dir=/backups/full-$(date +%F) \
  --user=backup --password='B@ckupPass'

# Prepare (apply logs) then it is restore-ready
mariabackup --prepare --target-dir=/backups/full-2026-07-13

# INCREMENTAL backup (based on the last full)
mariabackup --backup --incremental-basedir=/backups/full-2026-07-13 \
  --target-dir=/backups/incr-1 --user=backup --password='B@ckupPass'
```

BASH

```
# RESTORE (server stopped, datadir empty)
sudo systemctl stop mariadb
mariabackup --copy-back --target-dir=/backups/full-2026-07-13
sudo chown -R mysql:mysql /var/lib/mysql
```

```
sudo systemctl start mariadb
```

11.3 Logical backup & restore

BASH

```
# Consistent logical backup of one database
mariadb-dump --single-transaction --routines --triggers --events \
  --databases appdb -u backup -p > appdb-$(date +%F).sql

# Restore
mariadb -u root -p appdb < appdb-2026-07-13.sql
```

11.4 Point-in-time recovery (PITR)

1. Restore the most recent full (and incrementals) with mariabackup --copy-back.
2. Identify the recovery target time or binlog position from the incident.
3. Replay binary logs up to that point with `mariadb-binlog`.

BASH

```
# Replay events up to just before the bad change
mariadb-binlog --stop-datetime="2026-07-13 14:29:59" \
  /var/lib/mysql/binlog.000045 | mariadb -u root -p
```

✓ BEST PRACTICE — 3-2-1 backup rule

Keep 3 copies of data, on 2 different media, with 1 copy offsite/immutable. Regularly TEST restores — an untested backup is only a hypothesis.

12. Disaster Recovery

Disaster recovery (DR) protects against site-level failure. It is defined by two objectives: **RPO** (Recovery Point Objective — tolerable data loss) and **RTO** (Recovery Time Objective — tolerable downtime). Choose a strategy that meets both at acceptable cost.

12.1 DR strategies by RPO/RTO

Strategy	Typical RPO	Typical RTO	Cost
Backup + offsite restore	Hours	Hours	Low
Async replication to DR site	Seconds–minutes	Minutes	Medium
Delayed replica (safety net)	By design (e.g. 1h)	Minutes	Medium
Galera + geo-distributed	~0 (sync)	Seconds–minutes	High

12.2 Reference DR topology

◆ DIAGRAM — Primary + DR region with delayed safety replica

```
flowchart LR
  subgraph Primary[Primary Region]
    P[(Primary)]
    R[(Local Replica)]
    P --> R
  end
  subgraph DR[DR Region]
    D[(DR Replica\nasync)]
    DL[(Delayed Replica\n+1h)]
  end
  P == async WAN ==> D
  P == async WAN ==> DL
  D -. promote on disaster .-> APP[Applications re-point]
  BK[(Offsite backups)] --- P
```

12.3 Failover runbook (outline)

4. Declare disaster; confirm primary region is unrecoverable within RTO.
5. Verify DR replica caught up (`SHOW REPLICA STATUS`): low/zero lag.
6. Stop replication and promote: `STOP REPLICA; RESET REPLICA ALL;` set read_write.
7. Re-point application connection strings / DNS / MaxScale to the promoted node.
8. Rebuild a new replica behind the promoted primary; resume backups.

⛔ CRITICAL — Test DR regularly

A DR plan that has never been rehearsed will fail under real pressure. Schedule game-days: promote the DR replica, validate the application, and measure actual RTO/RPO against targets.

13. High Availability

High availability (HA) keeps the database serving through node failures. MariaDB offers asynchronous / semi-synchronous replication, synchronous multi-master Galera Cluster, and MaxScale for routing and automatic failover.

13.1 HA options compared

Approach	Model	Consistency	Best for
Async replication	Primary → replica(s)	Eventual	Read scaling, DR
Semi-sync replication	Primary waits for 1 ack	Stronger	Reduced data-loss risk
Galera Cluster	Synchronous multi-master	Strong (certification)	Write-anywhere HA
MaxScale + repl.	Proxy w/ auto-failover	Depends on backend	Transparent failover

13.2 Asynchronous replication setup

SQL

```
-- On PRIMARY: create a replication user
CREATE USER 'repl'@'10.0.0.%' IDENTIFIED BY 'ReplP@ss';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'10.0.0.%';
SHOW MASTER STATUS;   -- note File & Position (or use GTID)

-- On REPLICA: point at the primary and start
CHANGE MASTER TO
  MASTER_HOST='10.0.0.10', MASTER_USER='repl',
  MASTER_PASSWORD='ReplP@ss', MASTER_USE_GTID=slave_pos;
START REPLICA;
SHOW REPLICA STATUS\G
```

✓ BEST PRACTICE — Prefer GTID-based replication

Global Transaction IDs (MASTER_USE_GTID) make failover and replica repositioning far simpler than file+position coordinates, and are the modern default for MariaDB replication.

13.3 Galera Cluster + MaxScale

◆ DIAGRAM — Galera synchronous cluster behind MaxScale

```
flowchart TB
  APP[Applications] --> MX[MaxScale\nread/write split · failover]
  subgraph Galera[Galera Cluster - synchronous, 3 nodes]
    N1[(Node 1)]
    N2[(Node 2)]
    N3[(Node 3)]
    N1 <--> N2
    N2 <--> N3
    N1 <--> N3
  end
  MX -->|writes| N1
  MX -->|reads| N2
  MX -->|reads| N3
  note[Quorum needs majority\n3 nodes tolerate 1 failure]
```

INI

```
# /etc/my.cnf.d/galera.cnf  (each node)
[galera]
wsrep_on                = ON
wsrep_provider          = /usr/lib/galera/libgalera_smm.so
wsrep_cluster_name      = "sdc_cluster"
wsrep_cluster_address   = "gcomm://10.0.0.11,10.0.0.12,10.0.0.13"
wsrep_node_address      = "10.0.0.11"
wsrep_sst_method        = mariabackup
binlog_format           = ROW
default_storage_engine  = InnoDB
innodb_autoinc_lock_mode = 2
```

BASH

```
# Bootstrap the FIRST node only, then start the rest normally
galera_new_cluster                # node 1
sudo systemctl start mariadb      # nodes 2 & 3
# Verify cluster size
mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
```

13.4 Quorum & split-brain

> ⚠ **WARNING — Use an odd number of nodes**
> Galera requires a **majority quorum** to accept writes. Run **3 or 5 nodes** (odd) so the cluster tolerates failures without split-brain. For 2-node setups add a lightweight `garbd` arbitrator to break ties.

14. Troubleshooting

Troubleshoot methodically: reproduce, read the error log first, isolate the layer (client, network, SQL, engine, OS), form a hypothesis, change one thing, and verify. The error log is almost always the fastest starting point.

◆ DIAGRAM — Troubleshooting decision flow

```
flowchart TD
  S([Symptom reported]) --> L[Read error log\n/var/log/mariadb/error.log]
  L --> Q1{Server running?}
  Q1 -->|No| ST[Startup failure\nconfig / permissions / disk]
  Q1 -->|Yes| Q2{Can clients connect?}
  Q2 -->|No| NET[Auth / firewall / max_connections]
  Q2 -->|Yes| Q3{Slow or errors?}
  Q3 -->|Slow| PERF[EXPLAIN · SHOW PROCESSLIST\nINNODB STATUS]
  Q3 -->|Errors| ERR[Map error code\nsee Common Errors]
  Q3 -->|Replication| REP[SHOW REPLICA STATUS\nLast_Error]
```

14.1 Symptom → diagnosis → resolution

Symptom	Likely cause	Resolution
Won’t start	Bad config / perms / disk full / corrupt redo	Check error log; fix cnf; free space; innodb recovery
Can’t connect	Firewall / bind_address / max_connections	Open 3306; check bind; raise limit; verify user host
High CPU	Missing indexes / bad plans / scans	EXPLAIN; add indexes; kill runaway queries
High I/O wait	Small buffer pool / checkpoint storms	Increase buffer pool; tune io_capacity/log size
Replication stopped	Duplicate key / missing row / DDL drift	Inspect Last_Error; fix data; skip or re-sync
Deadlocks	Conflicting lock order	SHOW ENGINE INNODB STATUS; reorder access; retry logic
Disk full	Binlogs / temp / ibdata growth	Purge binlogs; expand volume; check tmpdir

BASH

```
# Fast triage kit
sudo tail -n 100 /var/log/mariadb/error.log
mariadb -e "SHOW FULL PROCESSLIST;"
mariadb -e "SHOW ENGINE INNODB STATUS\G" | sed -n '/DEADLOCK/,/TRANSACTIONS/p'
df -h /var/lib/mysql
mariadb -e "SHOW GLOBAL STATUS LIKE 'Threads_running';"
```

15. Common Errors

A reference of frequently encountered MariaDB errors with their root cause and remediation.

Code / message	Cause	Fix
1045 Access denied	Wrong user/host/password or auth plugin mismatch	Verify user@host & privileges; check auth plugin
2002 / 2003 Can't connect	Server down / socket wrong / firewall	Start service; check socket & bind_address; open 3306
1146 Table doesn't exist	Wrong DB, case sensitivity, or dropped table	Check lower_case_table_names; verify schema
1062 Duplicate entry	Unique/PK constraint violation	Deduplicate; use INSERT … ON DUPLICATE KEY
1205 Lock wait timeout	Long-held locks / contention	Shorten transactions; tune innodb_lock_wait_timeout
1213 Deadlock found	Circular lock dependency	Retry transaction; consistent access order
1114 Table is full	tmpdir/tablespace/disk limit	Free disk; raise tmp_table size; check quotas
1040 Too many connections	max_connections exhausted	Raise limit; use pooling; kill sleepers
1236 binlog read error	Purged/missing binlog on primary	Re-provision replica from fresh backup
WSREP: node dropped	Galera quorum loss / network split	Restore majority; check 4567; rejoin/SST
126 / 144 (corruption)	MyISAM/InnoDB page corruption	Restore from backup; mariadb-check; innodb recovery

⚠ WARNING — innodb_force_recovery is a last resort

Use `innodb_force_recovery` (1–6) only to dump data out of a corrupt instance, then rebuild. Never run a production server with it enabled, and never jump straight to level 6 — escalate one level at a time.

16. Frequently Asked Questions

Q1. Is MariaDB a true drop-in replacement for MySQL?
A. For MySQL 5.5–5.7 workloads, usually yes — same protocol, SQL, and clients. For MySQL 8.0+ specific features, validate compatibility first, as the projects have diverged.

Q2. What is the default storage engine?
A. InnoDB for user data (transactional, ACID). Aria is used for internal/system and temporary tables.

Q3. Is Galera synchronous?
A. Galera uses synchronous certification-based replication: a transaction is certified across the cluster before commit, giving effectively synchronous, virtually-lossless multi-master writes.

Q4. How do I upgrade MariaDB safely?
A. Back up, upgrade packages to the next major series, restart, then run mariadb-upgrade to fix system tables. Always test in staging and check release notes for behavior changes (e.g. snapshot isolation default in 12.3).

Q5. Which version should I run in production?
A. The latest LTS (12.3 as of mid-2026), or a still-supported LTS like 11.8 / 11.4. Avoid rolling releases and EOL series for production.

Q6. How do I reset a lost root password?
A. Start with --skip-grant-tables (or use unix_socket auth), then ALTER USER 'root'@'localhost' IDENTIFIED BY '...'; FLUSH PRIVILEGES; and restart normally.

Q7. Does MariaDB support JSON and vector search?
A. Yes — JSON functions are long-standing; native JSON improved in 11.x, and a native VECTOR type with HNSW indexing arrived in 11.8 for AI/similarity workloads.

Q8. What is the license?
A. The Community server is GPLv2 (fully open source). Connectors are LGPL, making them friendly for commercial application code.

Q9. How is MariaDB different from PostgreSQL?
A. MariaDB uses threads (not processes) and has deep, built-in replication/clustering. PostgreSQL emphasizes extensibility and strict standards compliance. Choice depends on workload and ecosystem.

Q10. Can I mix MariaDB and MySQL in replication?
A. Replicating MySQL → MariaDB is often possible one-way, but MariaDB → MySQL is generally unsupported. Test carefully; it is not recommended long-term.

17. Interview Questions

Curated questions with concise model answers, organized by theme — useful for DBA screening and cloud/data-engineering interview prep.

17.1 Fundamentals & architecture

1. Why was MariaDB created, and how does it relate to MySQL?
It is a community fork of MySQL started by MySQL's original author after Oracle's acquisition, guaranteeing an open-source, MySQL-compatible database with additional engines and features.

2. Explain MariaDB's pluggable storage-engine architecture.
A common SQL layer (parser/optimizer/executor) talks to storage engines through a handler API, so engines like InnoDB, Aria, ColumnStore, and MyRocks can be chosen per table without changing SQL.

3. Threads vs. processes — which does MariaDB use and why?
Threads (one per connection, optionally pooled), reducing memory and context-switch overhead versus process-per-connection designs.

4. What are the key InnoDB memory structures?
Buffer pool (caches pages), log buffer, change buffer, and adaptive hash index; on disk there are tablespaces, redo, and undo logs.

17.2 Replication & HA

1. Difference between asynchronous and semi-synchronous replication?
Async commits on the primary without waiting for replicas (risk of loss on failover); semi-sync waits for at least one replica to acknowledge receipt, reducing data-loss risk.

2. What problem do GTIDs solve?
Global Transaction IDs uniquely identify transactions cluster-wide, making failover and replica repositioning reliable without tracking binlog file/position manually.

3. How does Galera achieve consistency?
Certification-based synchronous replication: each transaction is broadcast and certified for conflicts across nodes before commit, giving virtually synchronous multi-master.

4. How many nodes should a Galera cluster have and why?
An odd number (3 or 5) to maintain majority quorum and avoid split-brain; 3 nodes tolerate one failure.

17.3 Performance & operations

1. What is the single most impactful performance setting?
innodb_buffer_pool_size — sized to ~60–80% of RAM on a dedicated host to keep the working set in memory.

2. How do you diagnose a slow query?
Capture it via the slow query log, run EXPLAIN/ANALYZE to inspect the plan, then add/fix indexes or rewrite; re-measure.

3. What does innodb_flush_log_at_trx_commit control?

Redo-log flush behavior on commit: 1 = full durability (flush every commit), 2 = flush to OS each commit (risk on OS crash), 0 = flush ~once/sec (highest throughput, most risk).

4. How do you perform point-in-time recovery?

Restore the latest full (plus incrementals), then replay binary logs up to the desired timestamp/position with mariadb-binlog.

17.4 Security & backup

1. How do you enforce encrypted client connections?

Configure server TLS (ssl_cert/key/ca), set require_secure_transport=ON, and/or REQUIRE SSL per user.

2. Difference between mariabackup and mariadb-dump?

mariabackup is a hot physical backup (fast, whole-instance, supports incrementals); mariadb-dump is a logical/SQL backup (portable, per-object, slower to restore large data).

3. What is the 3-2-1 backup rule?

Three copies of data, on two media types, with one offsite/immutable copy — and restores must be tested regularly.

4. How would you implement least-privilege access?

Create role-based grants scoped to specific schemas/objects, assign roles to users, avoid wildcard hosts and SUPER, and audit with server_audit.

18. Real-world Use Cases

MariaDB spans lightweight web workloads to large regulated and analytical systems. Representative scenarios:

Use case	Why MariaDB fits	Typical setup
LAMP / CMS (WordPress, Drupal)	Default DB in most distros; MySQL-compatible	Single node or primary + read replica
OLTP application backend	ACID InnoDB, strong concurrency	Primary + replicas + MaxScale
MySQL migration / modernization	Drop-in compatibility, open license	Cutover from MySQL with minimal change
High-availability SaaS	Galera synchronous multi-master	3-node Galera + MaxScale
Analytics / reporting	ColumnStore columnar engine	ColumnStore alongside OLTP
Write-heavy / space-constrained	MyRocks LSM engine	MyRocks tables for logging/IoT
Regulated (finance/health)	TLS, at-rest encryption, audit plugin	Encrypted cluster + SIEM + PITR
Edge / embedded	Small footprint, easy packaging	Single node with periodic backup

ℹ NOTE — Notable adopters

MariaDB is used by Wikipedia, is shipped as the default MySQL-compatible database across major Linux distributions, and has been adopted internally by large technology organizations — evidence of its production maturity at scale.

19. Best Practices

A consolidated operational checklist across the lifecycle. Treat these as defaults and adapt to your environment.

Deployment & configuration

- Run a supported LTS; standardize versions across environments; test upgrades in staging.
- Set `innodb_buffer_pool_size` to ~60–80% of RAM on dedicated hosts; keep configs in version control.
- Tune the OS: XFS/ext4, low swappiness, THP disabled, raised file-descriptor limits.

Security

- Always run mariadb-secure-installation; remove anonymous users and the test DB.
- Enforce least privilege via roles; avoid wildcard hosts and SUPER; disable remote root.
- Encrypt in transit (TLS) and at rest; enable server_audit; ship logs to a SIEM.

Reliability & recovery

- Enable binary logging with ROW format; use GTID-based replication.
- Layer physical + logical backups + binlog archiving; follow 3-2-1; test restores routinely.
- Design HA with an odd Galera node count or replicas behind MaxScale; rehearse DR failover.

Observability & operations

- Monitor buffer-pool hit ratio, connections, replication lag, locks, and disk headroom with alerts.
- Keep the slow query log on with a sensible threshold; review the top digests weekly.
- Automate patching and backup verification; document runbooks and keep them current.

> ✓ **BEST PRACTICE — Golden rule**
> Automate the routine, alert on the exceptional, and test the recovery path. Databases fail rarely but expensively — preparation is the differentiator.

20. References

Authoritative sources for further reading and current release information (verify version-specific details against the latest documentation).

[1] MariaDB Server Documentation (Knowledge Base) — https://mariadb.com/docs

[2] MariaDB Foundation — project & releases — https://mariadb.org

[3] MariaDB Server — all releases — https://mariadb.org/mariadb/all-releases/

[4] Community Server release notes — https://mariadb.com/docs/release-notes/community-server

[5] Galera Cluster documentation — https://galeracluster.com/library/

[6] MariaDB MaxScale documentation — https://mariadb.com/docs/maxscale

[7] mariabackup reference — https://mariadb.com/docs/server/server-usage/backup-and-restore/mariabackup

[8] MariaDB support lifecycle (EOL dates) — https://endoflife.date/mariadb

— End of Knowledge Base —

Document SDC-KB-MARIADB-001 · v1.0